JAVA PROGRAMMING / DSA

Chapter 39: Monotonic Stack & Deque

Personal Notes

"For each element, find the next one that's greater." "Find the largest rectangle in a histogram." "How much water gets trapped between bars?" These problems all seem to want an $O(n^2)$ nested-loop solution — but there's a beautiful O(n) pattern that solves them cleanly: the MONOTONIC STACK.

A monotonic stack is just a regular stack — but you enforce that the values inside stay in a particular order (increasing or decreasing). Elements that violate the order get POPPED. This tiny rule turns brute-force $O(n^2)$ problems into elegant O(n) sweeps. Once you spot the pattern, it becomes one of the highest-value tools in your DSA toolkit.

1. What is a Monotonic Stack?

A monotonic stack is a stack whose elements are always MONOTONIC — either strictly increasing or strictly decreasing from bottom to top. Whenever you're about to push a value that would break the order, you POP elements first until pushing is safe.

```
Increasing monotonic stack     Decreasing monotonic stack

   [1, 3, 5, 8]  (top)            [8, 5, 3, 1]  (top)

   Always increasing              Always decreasing
   bottom → top                    bottom → top

When pushing a new value:
   if it violates the order, POP until safe, then push.
```

Example: Pushing into an INCREASING Stack

```
Insert sequence: 2, 5, 3, 7, 1

push 2:  [2]                        stack is increasing ✓
push 5:  [2, 5]                     still increasing ✓
push 3:  [2, 5]  → pop 5 → [2, 3]   3 is smaller, pop then push
push 7:  [2, 3, 7]                  fine — increasing ✓
push 1:  [2, 3, 7] → pop 7 → pop 3 → pop 2 → [1]

Every element is pushed and popped AT MOST ONCE → O(n) total work.
```

Why it's O(n) despite the inner pop loop: Each element enters the stack once, and leaves at most once. So even though there's a while-loop inside a for-loop, the total number of push+pop operations is at most 2n. The amortized cost per element is O(1).

2. When to Reach for a Monotonic Stack

The moment a problem mentions any of these, monotonic stack is likely the answer:

- Next GREATER element to the right (or left)
- Next SMALLER element to the right (or left)
- Previous greater / smaller element
- For each element, how many elements until something bigger appears?
- Largest rectangle / area under a histogram-like shape
- How much water is trapped between bars?
- Any "span" problem where you need to look for a certain condition in the future

The pattern is the trigger: Whenever brute force gives you $O(n^2)$ with a nested loop searching "the next thing bigger/smaller," the monotonic stack is almost certainly the $O(n)$ upgrade. Learn to recognize the signal — it saves a lot of time in interviews.

3. Increasing vs Decreasing — Which One Do I Use?

Choose based on what you're LOOKING FOR:

If you want...	Use
Next GREATER element for each i	**Decreasing (pop smaller/equal values)**
Next SMALLER element for each i	**Increasing (pop larger/equal values)**
Previous GREATER element	**Decreasing (scanning left to right)**
Previous SMALLER element	**Increasing (scanning left to right)**
Largest rectangle in histogram	**Increasing (find left+right smaller bounds)**
Trapping rain water	**Decreasing (each pop identifies a valley)**
Sliding window MAXIMUM	**Decreasing deque (top is always max)**
Sliding window MINIMUM	**Increasing deque (top is always min)**

One-line rule of thumb: If you're looking for something GREATER → use DECREASING stack. If you're looking for something SMALLER → use INCREASING stack. The reason: you want to POP elements that are "in the way" of finding the target.

4. The Universal Template

Almost every monotonic stack solution fits this shape. Once you memorize it, the specific problem is just fill-in-the-blank.

```
Deque<Integer> stack = new ArrayDeque<>();   // store INDICES, not values

for (int i = 0; i < n; i++) {
    // Pop everything that violates the order
    while (!stack.isEmpty() && violatesOrder(stack.peek(), i)) {
        int top = stack.pop();
        // Do something with 'top' — it just found its answer!
        // (arr[i] is the next greater/smaller for arr[top])
    }

    stack.push(i);
}

// Elements remaining in the stack never found their answer
// — they get a default (usually -1 or n)
```

Three key decisions when adapting the template:

1. What ORDER are you maintaining? (Increasing or decreasing values)
2. What do you record when you POP? (The answer for the popped element)
3. What do the LEFTOVERS in the stack mean? (No such element exists → default)

> **Why store INDICES:** Storing indices gives you both the value (arr[i]) AND the position — which is essential for distance-based problems like Daily Temperatures. If you only need values, index-based code still works — just look up arr[i] when needed.

5. Classic Problem 1 — Next Greater Element

For each element in an array, find the FIRST element to its right that's greater. Return -1 if none exists.

Example

```
arr:    [2, 1, 2, 4, 3]
result: [4, 2, 4, -1, -1]

- 2 at index 0: next greater is 4
- 1 at index 1: next greater is 2
- 2 at index 2: next greater is 4
- 4 at index 3: no greater to the right → -1
- 3 at index 4: no greater to the right → -1
```

Code — Decreasing Monotonic Stack

```
public int[] nextGreater(int[] arr) {
```

```
    int n = arr.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);

    Deque<Integer> stack = new ArrayDeque<>();   // stores indices

    for (int i = 0; i < n; i++) {
        // While current is greater than stack top, it's the answer for
that index
        while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
            result[stack.pop()] = arr[i];
        }
        stack.push(i);
    }

    return result;
}

// Time: O(n), Space: O(n)
```

Dry Run for [2, 1, 2, 4, 3]

```
     i=0 (val=2): stack empty. push 0.   stack=[0]  result=[-1,-1,-1,-1,-
1]
     i=1 (val=1): arr[0]=2 > 1, no pop.  push 1.    stack=[0,1]
     i=2 (val=2): arr[1]=1 < 2 → pop 1, result[1]=2
                  arr[0]=2 not < 2, stop.  push 2.  stack=[0,2]
                  result=[-1, 2, -1, -1, -1]
     i=3 (val=4): pop 2, result[2]=4
                  pop 0, result[0]=4
                  push 3.  stack=[3]
                  result=[4, 2, 4, -1, -1]
     i=4 (val=3): arr[3]=4 > 3, no pop.  push 4.   stack=[3,4]

     After the loop: 3 and 4 stay in the stack → they get -1 (default)

     Final: [4, 2, 4, -1, -1]
```

6. Classic Problem 2 — Daily Temperatures

Given daily temperatures, for each day return the number of days you must wait until a WARMER temperature. If none exists, return 0.

Example

```
     temps:  [73, 74, 75, 71, 69, 72, 76, 73]
     answer: [ 1,  1,  4,  2,  1,  1,  0,  0]

     Day 0 (73): wait 1 day for 74
```

```
    Day 2 (75): wait 4 days for 76
    Day 6 (76): no warmer day → 0
```

Code

```
public int[] dailyTemperatures(int[] temps) {
    int n = temps.length;
    int[] result = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temps[stack.peek()] < temps[i]) {
            int idx = stack.pop();
            result[idx] = i - idx;    // days between = distance
        }
        stack.push(i);
    }

    return result;
}
```

Almost identical to Next Greater — but instead of storing the VALUE, store the DISTANCE. Same pattern, tiny tweak.

7. Classic Problem 3 — Largest Rectangle in Histogram

Given an array of heights representing histogram bars, find the area of the LARGEST RECTANGLE that fits inside the histogram. This is one of the most-asked Google interview problems — and the monotonic stack solution is O(n).

The Key Insight

For each bar, imagine extending a rectangle as WIDE as possible while keeping the height equal to that bar's height. The rectangle can extend LEFT and RIGHT until it hits a shorter bar. Then: area = height × (right - left - 1).

```
    heights = [2, 1, 5, 6, 2, 3]

                  ▓▓
            ▓▓    ▓▓
      ▓▓    ▓▓    ▓▓  ▓▓
      ▓▓    ▓▓    ▓▓  ▓▓
    0  1  2  3  4  5

    For bar with height 5 (index 2):
      - extends LEFT until index 1 (which is smaller)
      - extends RIGHT until index 4 (which is smaller)
      - width = 4 - 1 - 1 = 2
      - area  = 5 × 2 = 10  ← the answer!
```

```
     Best area explored: 10 (5×2 or 2×6 = 12 or 5×2 = 10 — actually 5×2
gives 10, but 2×6 doesn't
     apply here; the largest rectangle is 10 with bars at indices 2,3 both
at least 5 tall).
```

Code — Increasing Monotonic Stack

```
public int largestRectangleArea(int[] heights) {
    int n = heights.length;
    Deque<Integer> stack = new ArrayDeque<>();
    int maxArea = 0;

    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];   // sentinel to flush the
stack

        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int top = stack.pop();
            int height = heights[top];
            int width = stack.isEmpty() ? i : (i - stack.peek() - 1);
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }

    return maxArea;
}
```

The sentinel trick: By pretending there's a bar of height 0 at index n, we force the loop to pop everything remaining in the stack — meaning every real bar gets processed. Without the sentinel, bars still in the stack at the end never get to compute their area. This one-line trick saves an extra cleanup loop.

8. Classic Problem 4 — Trapping Rain Water

Given an array of heights representing an elevation map, compute how much water is trapped after it rains.

Example

```
     heights = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

             □ □ □   □ □   □   3
             □ □ □ ▪ □ □ □ □   2
       ▪ □ ▪ □ □ ▪ □ □ □ □ □   1
       ▪ ▪ ▪ ▪ ▪ ▪ ▪ ▪ ▪ ▪ ▪ ▪
     0 1 2 3 4 5 6 7 8 9 10 11
```

```
     Total water trapped = 6 units.

     Blocks marked□ are trapped water; ▪ are the bars themselves.
```

Code — Decreasing Monotonic Stack

```
public int trap(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int water = 0;

    for (int i = 0; i < heights.length; i++) {
        while (!stack.isEmpty() && heights[stack.peek()] < heights[i]) {
            int bottom = stack.pop();
            if (stack.isEmpty()) break;
            int left = stack.peek();
            int width = i - left - 1;
            int boundedHeight = Math.min(heights[left], heights[i]) -
heights[bottom];
            water += width * boundedHeight;
        }
        stack.push(i);
    }

    return water;
}
```

Every time you pop, you've found a VALLEY: heights[bottom] is the low point, bounded on the left (stack.peek() after pop) and right (heights[i]). The water above the bottom is width × (min(left, right) - bottom).

9. Monotonic Deque — Sliding Window Maximum

For sliding window problems (Ch 35), a monotonic DEQUE (double-ended queue) tracks the MAX or MIN as the window moves. Since values can leave the window from EITHER end, a regular stack won't work — you need to pop from BOTH ends.

Problem

Given an array and window size K, return the MAX of each window of K consecutive elements.

Example

```
     nums = [1, 3, -1, -3, 5, 3, 6, 7],  k = 3

     Window                Max
     [1, 3, -1]            3
     [3, -1, -3]           3
     [-1, -3, 5]           5
     [-3, 5, 3]            5
     [5, 3, 6]             6
```

```
 [3, 6, 7]            7

 Answer: [3, 3, 5, 5, 6, 7]
```

The Strategy

- Maintain a DECREASING deque of INDICES
- The FRONT of the deque is always the current window's maximum
- Before adding a new index, pop from the BACK any smaller values (they can never be max again)
- Also pop from the FRONT if that index has slid out of the window

Code

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();   // stores indices

    for (int i = 0; i < n; i++) {
        // Remove indices from BACK that are smaller than nums[i]
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) {
            dq.pollLast();
        }
        dq.offerLast(i);

        // Remove FRONT if it's out of the window
        if (dq.peekFirst() <= i - k) {
            dq.pollFirst();
        }

        // Record window max once we have a full window
        if (i >= k - 1) {
            result[i - k + 1] = nums[dq.peekFirst()];
        }
    }

    return result;
}

// Time: O(n) — each index enters and leaves the deque at most once.
```

Stack vs Deque: Both are monotonic. A STACK only pops from one end (top). A DEQUE can pop from BOTH ends. Sliding window problems need deques because values expire from the front (as the window slides) AND the back (as new values dominate old smaller ones).

10. Previous Greater / Smaller Element

For each element, find the previous element that's greater (or smaller). Same pattern, just scan left-to-right and look at the stack top when pushing.

```
public int[] previousGreater(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        // Pop smaller or equal — they can't be "greater"
        while (!stack.isEmpty() && arr[stack.peek()] <= arr[i]) {
            stack.pop();
        }
        // Whatever's on top NOW is the previous greater
        result[i] = stack.isEmpty() ? -1 : arr[stack.peek()];
        stack.push(i);
    }

    return result;
}
```

Notice the difference: we DON'T record answers when popping. We record the answer for the CURRENT element by looking at the top AFTER popping smaller ones.

11. Circular Array Variants

Some problems have a CIRCULAR array — the "next greater" of the last element could wrap around to the beginning. The trick: iterate the array TWICE (or use i % n modulo indexing).

```
public int[] nextGreaterCircular(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();

    // Loop 2n times to simulate wrap-around
    for (int i = 0; i < 2 * n; i++) {
        int idx = i % n;
        while (!stack.isEmpty() && nums[stack.peek()] < nums[idx]) {
            result[stack.pop()] = nums[idx];
        }
        if (i < n) stack.push(idx);   // only push once per real index
    }

    return result;
}
```

12. Complexity Analysis

Problem	Time	Space
Next Greater / Smaller	O(n)	O(n) worst case
Daily Temperatures	O(n)	O(n)
Largest Rectangle in Histogram	O(n)	O(n)
Trapping Rain Water	O(n)	O(n)
Sliding Window Maximum	O(n)	O(k)
Circular Next Greater	O(n)	O(n)

The magic of monotonic stack: every element is pushed and popped at most once, so despite the nested while-loop, total work is O(n) — amortized.

13. Common Mistakes

Mistake 1: Storing values instead of indices

```
// If you need distances (Daily Temperatures) or bounds (Largest
Rectangle),
// storing values loses the position info.

// BAD if you need positions:
stack.push(arr[i]);

// GOOD — store indices, look up values via arr[idx]:
stack.push(i);
```

Mistake 2: Using Stack class instead of Deque

```
// Legacy — slower due to synchronization
Stack<Integer> stack = new Stack<>();

// Modern, faster:
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);     // same interface (push/pop/peek)
```

Mistake 3: Wrong direction of monotonic order

Increasing vs decreasing is easy to mix up. Remember the rule: looking for GREATER → DECREASING stack. Looking for SMALLER → INCREASING stack. When in doubt, work through a tiny example by hand.

Mistake 4: Off-by-one in width calculation

```
// Largest Rectangle — width when the stack is non-empty:
int width = i - stack.peek() - 1;    // ✓ correct

// Common mistake:
int width = i - stack.peek();        // ✗ counts one too many
```

Mistake 5: Forgetting the sentinel in histogram problem

Without appending a 0 at the end, bars still in the stack never get processed. Either add a 0-height sentinel, or add a separate flush loop after the main pass.

Mistake 6: Using < vs <= incorrectly

```
// For "next STRICTLY greater":
while (!stack.isEmpty() && arr[stack.peek()] < arr[i])

// For "next greater OR equal":
while (!stack.isEmpty() && arr[stack.peek()] <= arr[i])

// Read the problem carefully — small difference, different answer.
```

Mistake 7: Not clearing the deque of expired indices

In sliding window problems, if you forget to `pollFirst()` when the front index falls out of the window, you'll report a max from OUTSIDE the current window. Always check `dq.peekFirst() <= i - k` and pop if true.

14. Best Practices

- Use ArrayDeque, not Stack — faster and more idiomatic Java
- Store indices, not values — gives you positions AND values
- Decide upfront: increasing or decreasing? Match to what you're LOOKING FOR
- For largest-rectangle-style problems, use a 0 sentinel at the end
- For sliding window, use a Deque (poll from both ends)
- Draw a small example by hand before coding to lock in the pop logic
- Confirm your < vs ≤ — strict comparison changes semantics
- Trace the algorithm on 3-4 elements first — the pop rhythm reveals bugs quickly

15. Quick Reference

Concept	Meaning
Monotonic Stack	Stack whose values stay increasing or decreasing

Concept	Meaning
Monotonic Deque	Same idea but pops from both ends (for windows)
Increasing stack	Pop when incoming ≤ top — for "next smaller"
Decreasing stack	Pop when incoming ≥ top — for "next greater"
Store indices	Not values — indices preserve position
ArrayDeque	Preferred over Stack in Java
Amortized O(n)	Each index pushed/popped at most once
Sentinel	Extra 0 or Integer.MAX_VALUE to flush the stack
Next greater/smaller	Classic use case — $O(n^2) \rightarrow O(n)$
Largest histogram	Uses increasing stack + width formula
Trapping rain water	Decreasing stack — each pop identifies a valley
Sliding window max/min	Monotonic deque

16. Practice Problems

Practice in order — first the classics, then variants.

Foundational

4. Next Greater Element I (LeetCode 496).
5. Next Greater Element II — circular (LeetCode 503).
6. Next Greater Element III — nearest larger permutation (LeetCode 556).
7. Daily Temperatures (LeetCode 739).
8. Number of Visible People in a Queue (LeetCode 1944).

Histogram Family

9. Largest Rectangle in Histogram (LeetCode 84) — the classic.
10. Maximal Rectangle in a Binary Matrix (LeetCode 85).
11. Trapping Rain Water (LeetCode 42).
12. Sum of Subarray Minimums (LeetCode 907).
13. Sum of Subarray Ranges (LeetCode 2104).

Sliding Window with Monotonic Deque

14. Sliding Window Maximum (LeetCode 239).
15. Sliding Window Minimum.
16. Shortest Subarray with Sum at Least K (LeetCode 862).
17. Jump Game VI (LeetCode 1696) — monotonic deque + DP.

String Problems

18. Remove K Digits (LeetCode 402).
19. Remove Duplicate Letters (LeetCode 316).
20. Smallest Subsequence of Distinct Characters (LeetCode 1081).
21. 132 Pattern (LeetCode 456).

Advanced

22. Stock Span Problem — number of consecutive days before with lower price.
23. Constrained Subsequence Sum (LeetCode 1425).
24. Online Stock Span (LeetCode 901).
25. Car Fleet (LeetCode 853) — stack-based simulation.

17. Final Takeaway

Monotonic stack is one of those techniques that looks obscure at first but becomes second nature after 5-6 problems. The template is small: a stack, a for-loop, and a while-loop that pops elements out of order. What changes between problems is only what you RECORD when you pop.

For interviews, master four problems: Next Greater Element, Daily Temperatures, Largest Rectangle in Histogram, and Trapping Rain Water. Plus one deque variant: Sliding Window Maximum. Those five cover 80% of monotonic stack questions you'll ever face.

> **Key Takeaway:** Monotonic stack turns $O(n^2)$ "next greater/smaller" problems into O(n). Choose direction based on what you're looking for (greater → decreasing stack; smaller → increasing). Store indices, use ArrayDeque, and record answers WHEN YOU POP. For sliding window max/min, use a monotonic deque that pops from both ends. Master this pattern and a whole category of hard-looking problems becomes routine.

18. What's Next?

Monotonic stack was one of the last big DSA patterns. What remains:

- Two Pointers in depth — palindrome, 3-sum, container with most water
- Prefix Sums — subarray sums with negatives, range queries
- Greedy Algorithms — activity selection, interval scheduling

- Segment Tree / Fenwick Tree — range queries with point updates
- Advanced Strings — KMP, Rabin-Karp, Z-algorithm
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points
- System Design — the next big step in interview prep